

UNIT-1-INTRODUCTION TO SOFTWARE ENGINEERING

1.1 Program vrs Software product

1.2 Emergence of Software Engineering.

1.3 Computer Systems Engineering

1.4 Software Life Cycle Models

1.4.1 Classical Water fall model

1.4.2 Iterative Water fall model

1.4.3 Prototyping model

1.4.4 Evolutionary model

1.4.5 Spiral model

INTRODUCTION TO SOFTWARE:

- Software is a set of instructions used to get inputs and process them to produce the desired output.
- It also includes a set of documents, such as the software manual, Source Code, Executables, Design Documents, Operations, and System Manuals and Installation and Implementation Manuals.

Software Engineering

A few important definitions given by several authors and institutions are as follows:

Definition:

Software Engineering is the application of a systematic, disciplined, scientific approach to the development, operation and maintenance of software.

OR

Software Engineering is a discipline whose aim is the production of fault free software that satisfies the user's needs and that is delivered on time and within budget.

Goals of Software Engineering:

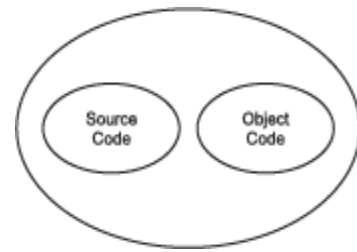
The primary goals of Software Engineering is

1. To improve the quality, reliability of software.
2. To increase productivity.
3. To increase the job satisfaction of software engineers, etc.

PROGRAMS VERSUS SOFTWARE PRODUCTS

Programs

- A program is a combination of source code and object code.
- A program is a subset of software and it becomes software only if documentation and operating procedure manuals are prepared.
- Program = Source Code + Object Code



Software Products

- A software product consists not only of the program code but also of all the associated documents, such as the requirements specification documents, the design documents, the test documents, and the operating procedures, user manuals and operational manuals, etc.
- Software = Program + Documentation + Operating Procedures



Programs versus Software Products

The various differences between a program product and a software product are given in Table.

S. No.	Programs	Software Products
1.	Programs are developed by individuals for their personal use.	A software product is usually developed by a group of engineers working as a team.
2	Usually small in size	Usually large in size
3	Single user	Large number of users
4	Single developer	Team of developers
5	Lack proper documentation	Good documentation support
6	Adhoc development	Systematic development
7	Lack of user interface	Good user interface
8	Have limited functionality	Exhibit more functionality

SOFTWARE-DEVELOPMENT LIFE-CYCLE

- The software-development life-cycle is used to help the development of a large software product in a systematic, well-defined, and cost-effective way.
- It goes through a series of phases from start to end. This process is called the Software-Development Life-Cycle.
- The software development life-cycle consists of several phases.
- These are:
 - Feasibility study
 - Project analysis (Requirements analysis and specification)
 - System design (Design)
 - Coding
 - Testing
 - Implementation
 - Maintenance

Software Development Life Cycle Models

- A software life cycle model (also called process model) is a descriptive and diagrammatic representation of the software life cycle.

Need for a software life cycle Model:

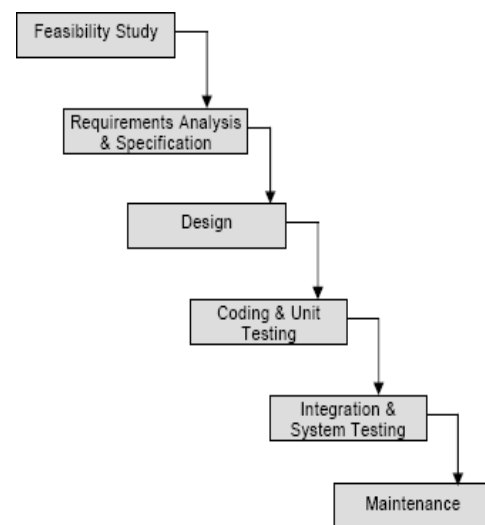
- The development team must identify a suitable life cycle model to develop a project..
- Without using of a particular life cycle model the development of a software product may lead to project failure.

Different software life cycle models:

- Various kinds of process models:
 - Classical Waterfall model
 - Iterative Waterfall model
 - Prototyping model
 - Incremental model
 - Spiral model

Classical waterfall model:

- The waterfall model is a very common and traditional SDLC model or software process model.
- The figure is given below.
- This model is known as the *waterfall model*, because of the cascade from one phase to another phase.
- As the figure shows, the phases are cascade in nature, where the output of one phase is the input to the next one. Each phase performs a set of activities.
- All other life cycle models are essentially derived from the classical waterfall model.
- Classical waterfall model has the following phases.
 - Feasibility Study
 - Requirements Analysis and Specification
 - Design
 - Coding and Unit Testing
 - Integration and System Testing, and
 - Maintenance



Feasibility study:

The main aim of feasibility study is to determine whether the project is **financially and technically feasible or not**, to develop the product.

- At first project managers or team leaders try to have a rough understanding of the project by visiting the client side. They study or discuss different aspects of project like resources required, cost, time for development etc.
- At the end of this phase, a report called a feasibility study is prepared by a group of software engineers.

Requirements analysis and specification:

The aim of the requirements analysis and specification phase is to understand the exact requirements of the customer and to document them properly. This phase consists of two distinct activities, namely

- Requirements gathering and analysis, and
- Requirements specification.
- The goal of the requirements gathering activity is to collect all the related information from the customer regarding the product to be developed.
- The final document is known as the software requirement specification (SRS) document.

Design:

The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation by using some programming language.

- Two distinctly different approaches are available:
 - Architectural Design approach.
 - Detailed Design approach.
- **Architectural Design:**
 - It involves:
 - Identifying the software components.
 - Decomposing the software components into modules.
 - Specifying the interconnection between the various components.
- **Detailed Design:**
 - It deals with all details for the implementation i.e. procedures to process the algorithms, data structures to be used, etc.
- **Output:**
 - Design Document
 - Test Plan

Coding and unit testing:

- The purpose of the coding and unit testing phase (sometimes called the implementation phase) of software development is to translate the software design into source code.
- The end of this phase, each unit module is tested against each unit test plan.
- Output of this phase is:
 - Program code or source code
 - Unit test report

Integration and System Testing:

- Integration of different modules is undertaken once they have been coded and unit tested.
- During the integration and system testing phase, the modules are integrated.
- System testing usually consists of three different kinds of testing activities:-
 - **α – testing:** It is the system testing performed by the development team.
 - **β – Testing:** It is the system testing performed by a friendly set of customers.
 - **Acceptance testing:** It is the system testing performed by the customer himself after the product delivery to determine whether to accept or reject the delivered product.
- Output:
 - Final test report
 - Total product

Maintenance:

- The product is ready and it is delivered to the client.
- Maintenance of a typical software product requires much more than the effort necessary to develop the product itself.
- The relative effort of development of a typical software product to its maintenance effort is roughly in the 40:60 ratio.
- Maintenance involves performing any one or more of the following three kinds of activities:
 - **corrective maintenance**
 - **perfective maintenance**
 - **adaptive maintenance**

Advantages of Waterfall Model

- It is a linear model.
- Relatively easy to understand and manage.
- It is systematic and sequential.
- It is a simple one.
- It has proper documentation.

Disadvantages of waterfall model:

- It is difficult to define all requirements at the beginning of a project.
- This model is not suitable for accommodating any change.
- It does not suitable to large projects.
- It involves heavy documentation.
- We cannot go backward in the SDLC.
- There is no risk analysis.

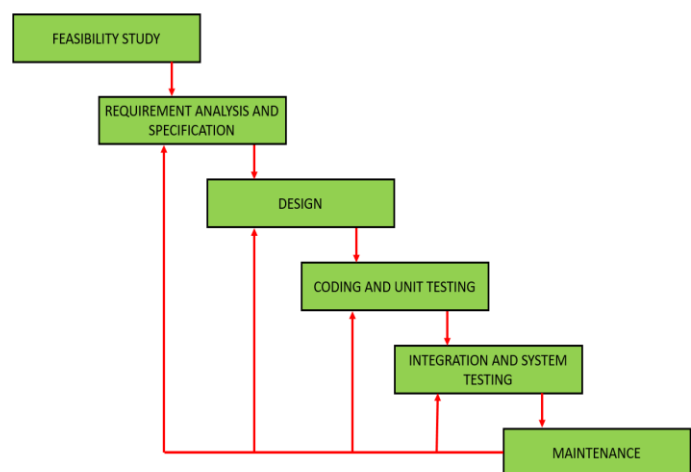
What is maintenance? Different types of maintenance. Or short note on maintenance.

Maintenance:

- The product is ready and it is delivered to the client.
- Maintenance of a typical software product requires much more than the effort necessary to develop the product itself.
- The relative effort of development of a typical software product to its maintenance effort is roughly in the 40:60 ratio.
- Maintenance involves performing any one or more of the following three kinds of activities:
 - **corrective maintenance**
 - **perfective maintenance**
 - **adaptive maintenance**
- Correcting errors that were not discovered during the product development phase. This is called **corrective maintenance**.
- Improving the implementation of the system, and enhancing the functionalities of the system according to the customer's requirements. This is called **perfective maintenance**.
- Porting the software to work in a new environment. For example, porting may be required to get the software to work on a new computer platform or with a new operating system. This is called **adaptive maintenance**.

Iterative Waterfall Model:

- Beside the weaknesses of the waterfall model, **Iterative development** is the heart of acyclic software development process.
- It starts with feasibility study and ends with deployment (maintenance) with the cyclic interactions in between phases. i.e. it provide feedback paths from every phase to its preceding phases.
- The feedback paths allow for correction of the errors done during a phase.



- The principle of detecting errors as close to its point of introduction as possible. This is known as “Phase Containment of Errors”.
- To achieve this review is done after completion of each phase.
- The rest of the phases of development are same as classical water fall model. The model is pictorially shown in fig.

Prototyping Model:

It sometimes happens that a customer defines a set of general objectives for software but does not identify detailed requirements (like input, processing, or output).

Or in other cases, the developer may not be sure about the efficiency of an algorithm, operating system, used etc. In these situations, a prototyping model may be used.

- A prototype is a toy implementation of the system.
- A prototype usually shows limited functional capabilities, low reliability, and inefficient performance.
- It uses dummy functions.
- A prototype is the partial version of the actual system.

Where to use Prototype Model:

A prototype of the actual product is preferred in situations such as:

- User requirements are not complete.
- Technical issues are not clear.

The prototyping model of software is shown in fig:

The phases of this model are:

Requirement Gathering:

- In this model a prototyping starts with an initial requirement and gathering phase.
- The functionalities required for the product are gathered.

Quick Design:

- A quick design is carried out and a prototype is build.
- A new plan is created from gathered requirements for the prototype to be built.

Prototype creation:

- Based on the design issues the trial product is created.

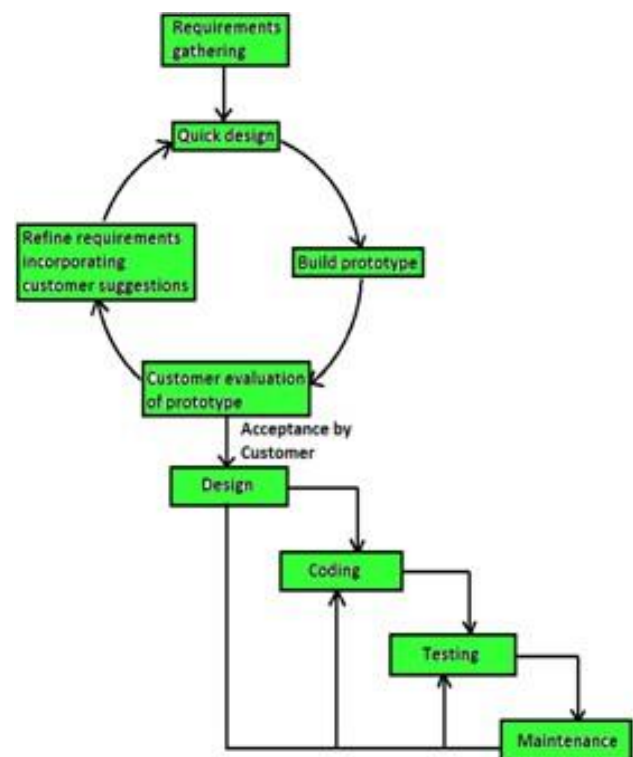
Customer Evaluation:

- The developed prototype is submitted to the customer for evaluation.
- Based on the customer feedback we may go for development or again go for the redesign as per customer demand.

Prototype refinement:

- Based on the information.

The code for the prototype is usually thrown away, but the experience gained while developing the prototype helps for actual development. The rest of the phases for development are similar to the waterfall model.



Advantages of Prototyping Models:

- Suitable for large systems.
- Customer communication is available.
- Quality of software is good.
- It helps to identifying the requirements.

Limitations of Prototyping Model:

- It is very difficult to predict how the system will work after development.
- This model is time consuming.

Incremental Model or Evolutionary Model:

This life cycle model is also referred to as the successive versions model and sometimes as the **incremental model**.

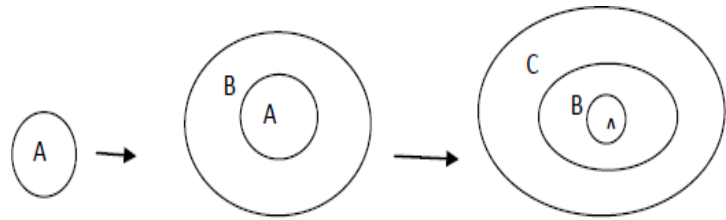


Fig: Evolutionary model (A,B,C are modules of the software model)

- In this lifecycle model the software is first broken down into several modules which can be incrementally constructed and delivered.
- The development team first develops the core modules of the system.
- The development model is shown in fig.
- Each successive version of the product is fully functioning capable of performing more useful work than the previous version.
- In this model the user gets a chance to experiment with particularly developed software much before the complete version of the software is ready.
- This model is useful only for very large products.

Advantages:

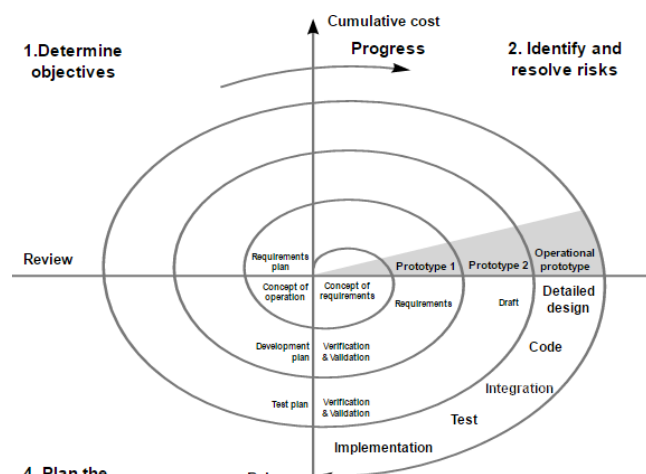
- Problem understanding increases through successive refinements
- Performs cost benefit analysis before enhancing software with capabilities
- Does not involve high complexity rate
- Early feedback is generated

Disadvantages

- Requires planning at the management and technical level.
- Becomes invalid when there is time constraint in the project schedule.

Spiral Model

- The spiral model, originally proposed by Barry Boehm.
- Main objective is to analyze the risks involved in the software project.
- Combines the features of prototyping model, evolutionary and iterative waterfall model. Hence this model is also called meta model.
- The diagrammatic representation of this model appears like a spiral with many loops. The exact number of loops in the spiral is not fixed.
- Each loop of the spiral represents a phase



of the software process. For example, the innermost loop might be feasibility study, the next loop may be requirements specification, and so on.

- Each phase in this model is split into four sectors (or quadrants) as shown in fig. The following activities are carried out during each phase of a spiral model.

First quadrant (Objective Setting)

- During the first quadrant, it determines objectives, alternatives & constraints.

Second Quadrant (Risk Assessment and Reduction)

- A detailed analysis is carried out for each identified project risk.
- Steps are taken to reduce the risks.

Third Quadrant (Development and Validation)

- Develop and validate the next level of the product after resolving the identified risks.

Fourth Quadrant (Review and Planning)

- Review the results achieved so far with the customer and plan the next iteration around the spiral.
- Progressively more complete version of the software gets built with each iteration around the spiral.

Advantages

- For large, complex and expensive projects.
- If the software associated with risk.
- Re-evaluation after each step

Disadvantages

- Assessment and resolution of project risks is not an easy task.
- Difficult to estimate budget and schedule in the beginning.

UNIT-2

Project Planning and Project Estimation Techniques

Responsibilities of a software project manager

- Software project managers take the overall responsibility of a project to success.
- It is very difficult to particularly describe the job responsibilities of a project manager.
- The job responsibility of a project manager ranges from building up team morale to highly visible customer presentations.
- Most managers take responsibility for project proposal writing, project cost estimation, scheduling, project staffing, software process tailoring, project monitoring and control, software configuration management, risk management, interfacing with clients, managerial report writing and presentations, etc.

Skills necessary for software project management

- A theoretical knowledge of different project management techniques is certainly necessary to become a successful project manager.
- However, effective software project management frequently calls for good qualitative judgment and decision taking capabilities.
- Should have good knowledge about of latest software.

Project planning

Once a project is found to be feasible, software project managers undertake project planning. Project planning is undertaken and completed before any development activity starts. Project planning consists of the following essential activities:

- Estimating the following attributes of the project:
 - Project size:** What will be problem complexity in terms of the effort and time required to develop the product?
 - Cost:** How much is it going to cost to develop the project?
 - Duration:** How long is it going to take to complete development?
 - Effort:** How much effort would be required?

Software Project Management Plan (SPMP)

- Once project planning is complete, project managers document their plans in a Software Project Management Plan (SPMP) document.
- The SPMP document should discuss a list of different items that have been discussed below.
 1. **Introduction** (Objectives, Major Functions, Performance Issues, Management and Technical Constraints).
 2. **Project Estimates** (Historical Data Used, Estimation Techniques Used, Effort, Resource, Cost, and Project Duration Estimates).
 3. **Schedule** (Work Breakdown Structure, Activity Network Representation, Gantt Chart Representation, PERT Chart Representation).
 4. **Project Resources** (People, Hardware and Software, Special Resources).
 5. **Staff Organization**
 6. **Risk Management Plan** (Risk Analysis, Risk Identification, Risk Estimation)
 7. **Project Tracking and Control Plan**
 8. **Miscellaneous Plans** (Quality Assurance Plan, Configuration Management Plan)

Sliding Window Planning

- Project planning is a very difficult for large projects to make accurate plans.
- Project planning requires utmost care and attention otherwise projects will failure.
- Schedule delays can cause customer dissatisfaction and affect the team morale.

- This happens due to the fact that the proper parameters, scope of the project, project staff, etc. may change during the span of the project.
- In order to overcome this problem, sometimes project managers undertake project planning in stages. This protects managers from making big commitments too early.
- This technique is known as Sliding Window Planning. In the sliding window technique, starting with an initial plan, then go for successive development stages.
- At the start of a project, project managers have incomplete knowledge about the details of the project. Their information base gradually improves as the project progresses through different phases.
- After the completion of every phase, the project managers can plan each subsequent phase more accurately and with increasing levels of confidence.

Metrics for software project size estimation

- In order to be able to accurately estimate the project size, some important metrics should be defined in terms of which the project size can be expressed.
- Currently two metrics are popularly being used widely to estimate size:
 - **Lines of Code (LOC)**
 - **Function Point (FP)**
- The usage of each of these metrics in project size estimation has its own advantages and disadvantages.

Lines of Code (LOC)

- LOC is the simplest, very popular among all metrics available to estimate project size.
- Using this metric, the project size is estimated by counting the number of source instructions/lines in the developed program.
 - While counting the number of source instructions, lines used for commenting the code and the header lines should be ignored.
- Determining the LOC count at the end of a project is a very simple job. But, at the beginning of a project is very difficult to accurately measure the LOC count.
- In order to estimate the LOC count at the beginning of a project, project managers usually divide the problem into modules and each module into sub modules and so on, until the sizes of the different leaf-level modules can be approximately predicted.
- To be able to do this, past experience in developing similar products is helpful.

Disadvantages of Using LOC

- Size can vary with coding style.
- Focuses on coding activity alone.
- Measures lexical / textual complexity only.
 - Does not address the issues of structural or logical complexity.
- Difficult to estimate LOC from problem description.

Function point (FP)

- Function point metric was proposed by Albrecht [1983].
 - This metric overcomes many of the shortcomings of the LOC metric.
 - By using the function point metric we can easily estimate the size of a software product directly from the problem specification.
 - This is in contrast to the LOC metric, where the size can be accurately determined only after the product has fully been developed.
 - The FP can be computed in two steps:
 - Unadjusted Function Point (UFP)
 - Technical complexity Factor (TFC)
 - Once the UFP is computed, the TFC is computed next.
- $FP = UFP + TFC$.

Project Estimation techniques

- The basic project planning activity is Estimation of various project parameters.
- The important project parameters are: project size, effort required to develop the software, project duration, and cost.
- These estimates not only help in quoting the project cost to the customer, but are also useful in resource planning and scheduling.
- There are three broad categories of estimation techniques:
 - **Empirical estimation techniques**
 - **Heuristic techniques**
 - **Analytical estimation techniques**

Empirical Estimation Techniques:

- This techniques based on making an educated guess of the project parameters.
- While using this technique, prior experience with similar products is helpful.
- Two popular empirical estimation techniques are:
 - Expert judgment technique
 - Delphi cost estimation.

Expert Judgment Technique

- It is one of the most widely used estimation techniques.
- In this approach, an expert makes an educated guess of the problem size after analyzing the problem thoroughly.
- Usually, the expert estimates the cost of the different components (i.e. modules or subsystems) of the system and then combines them to arrive at the overall estimate.

Delphi cost estimation

- This approach tries to overcome some of the shortcomings of the expert judgment approach.
- It is carried out by a team comprising of a group of experts and a coordinator.
- In this approach, the coordinator provides each estimator with a copy of the software requirements specification (SRS) document and a form for recording his cost estimate.
- Estimators complete their individual estimates anonymously and submit to the coordinator.
- In their estimates, the estimators mention any unusual characteristic of the product which has influenced his estimation.
- The coordinator prepares and distributes the summary of the responses of all the estimators, and includes any unusual rationale noted by any of the estimators. Based on this summary, the estimators re-estimate.
- This process is iterated for several rounds.
- However, no discussion among the estimators is allowed during the entire estimation process.
- The idea behind this is that if any discussion is allowed among the estimators, then many estimators may easily get influenced by the rationale of an estimator who may be more experienced or senior.
- After the completion of several iterations of estimations, the coordinator takes the responsibility of compiling the results and preparing the final estimate.

Heuristic Techniques:

- This technique assumes that the relationships among the different project parameters can be modeled using suitable mathematical expressions.
- Once the basic (independent) parameters are known, the other (dependent) parameters can be easily determined by substituting the value of the basic parameters in the mathematical expression.
- Different heuristic estimation models can be divided into the following two classes:
 - single variable model and
 - The multi variable model.

COCOMO Model:

- The Constructive Cost Model (COCOMO) is an heuristic estimation model i.e., the model uses a theoretically derived formula to predict cost related factors. This model was created by “Barry Boehm”. The COCOMO model consists of three models:-
- The COCOMO models are defined for three classes of software projects, stated as follows i.e. it divides software product developments into 3 categories:
 - **Organic**
 - **Semidetached**
 - **Embedded**

Organic:

- A development project can be considered of organic type, if
 - The project deals with developing a well understood application program.
 - The size of the development team is reasonably small.
 - The team members are experienced in developing similar types of projects.

Semidetached:

- A development project can be considered of semidetached type, if
 - The development consists of a mixture of experienced and inexperienced staff.
 - Team members may have limited experience on related System.

Embedded:

- A development project is considered to be of embedded type, if
 - The software being developed is strongly coupled to complex hardware.
- The COCOMO model consists of three models(i.e. Software cost estimation is done through three stages):
 - **Basic COCOMO,**
 - **Intermediate COCOMO,**
 - **Complete COCOMO.**

Basic COCOMO Model

- The basic COCOMO model gives an approximate estimate of the project parameters.
- The basic COCOMO estimation model is given by the following expressions:

$$\text{Effort} = a_1 * (\text{KLOC})^{a_2} \text{ PM}$$

$$T_{\text{dev}} = b_1 * (\text{Effort})^{b_2} \text{ Months}$$
 - Where
 - KLOC is the estimated size of the software product expressed in Kilo Lines of Code,
 - a_1, a_2, b_1, b_2 are constants for each category of software products,
 - T_{dev} is the estimated time to develop the software, expressed in months,
 - Effort is the total effort required to develop the software product, expressed in person months (PMs).
- **Estimation of development effort:**
 - For the three classes of software products, the formulas for estimating the effort based on the code size are shown below:
 - Organic : Effort = $2.4(\text{KLOC})^{1.05} \text{ PM}$**
 - Semi-detached : Effort = $3.0(\text{KLOC})^{1.12} \text{ PM}$**
 - Embedded : Effort = $3.6(\text{KLOC})^{1.20} \text{ PM}$**
- **Estimation of development time:**

- For the three classes of software products, the formulas for estimating the development time based on the effort are given below:

Organic : $T_{dev} = 2.5(\text{Effort})^{0.38}$ Months

Semi-detached : $T_{dev} = 2.5(\text{Effort})^{0.35}$ Months

Embedded : $T_{dev} = 2.5(\text{Effort})^{0.32}$ Months

Intermediate COCOMO model:

- The basic COCOMO model assumes that effort and development time are functions of the product size alone.
 - However, a host of other project parameters besides the product size affect the effort required to develop the product as well as the development time.
- Therefore, in order to obtain an accurate estimation of the effort and project duration, the effect of all relevant parameters must be taken into account.
 - This model recognizes this fact and refines the initial estimate obtained using the basic COCOMO expressions by using a set of 15 cost drivers (multipliers) based on various attributes of software development.
 - Boehm requires the project manager to rate these 15 different parameters for a particular project on a scale of one to three.
- **Example:**
 - If modern programming practices are used, the initial estimates are scaled downward by multiplication with a cost driver having a value less than 1.
 - If there are reliability requirements on the software product, this initial estimate is scaled upward.
- In general, the cost drivers can be classified as being attributes of the following items:

Product:

- It include complexity of the product, reliability requirements of the product, etc.

Computer:

- It include the execution speed required, storage space required etc.

Personnel:

- It includes the experience level of personnel, programming capability, analysis capability, etc.

Development Environment:

- Development environment attributes capture the development facilities available to the developers.
- Ex: automation (CASE) tools.

Complete COCOMO model:

- A major shortcoming of both the basic and intermediate COCOMO models is that they consider a software product as a single homogeneous entity.
- However, most large systems are made up several smaller sub-systems. These sub-systems may have widely different characteristics.
- For example, some sub-systems may be considered as organic type, some semidetached, and some embedded.
- Not only that the inherent development complexity of the subsystems

Analytical Estimation Techniques

- It derive the required results starting with basic assumptions regarding the project.
- Thus, unlike empirical and heuristic techniques, analytical techniques do have scientific basis. Halstead's software science is an example of an analytical technique.
- Halstead's software science can be used to derive some interesting results starting with a few simple assumptions.

Halstead's Software Science – An Analytical Technique:

- It is an analytical technique to measure size, development effort, and development cost of software products.
- Halstead used a few primitive program parameters to develop the expressions for overall program length, potential minimum value, actual volume, effort, and development time.
- For a given program, let:
 - η_1 be the number of unique operators used in the program,
 - η_2 be the number of unique operands used in the program,
 - N_1 be the total number of operators used in the program,
 - N_2 be the total number of operands used in the program.
- It derived expressions for:
 - over all program length,
 - potential minimum volume
 - actual volume,
 - language level,
 - effort, and
 - development time.
 - Staffing level estimation:
- Once the effort required to develop a software has been determined, it is necessary to determine the staffing requirement for the project.
- Putnam first studied the problem of what should be a proper staffing pattern for software projects. He extended the work of Norden who had earlier investigated the staffing pattern of research and development (R&D) type of projects.

Putnam's Work

- Putnam studied the problem of staffing of software projects and found that the software development has characteristics very similar to other R & D projects studied by Norden and that the Rayleigh-Norden curve can be used to relate the number of delivered lines of code to the effort and the time required to develop the project.
- By analyzing a large number of army projects, Putnam derived the following expression:

$$L = C_k K^{1/3} t_d^{4/3}$$

$$K = C/t_d^4$$

For the same product size, $C = L^3 / C_k^3$ is a constant.

or, $K_1/K_2 = t_{d2}^4/t_{d1}^4$

or, $K \propto 1/t_d^4$

or, $\text{cost} \propto 1/t_d$

(as project development effort is equally proportional to project development cost)

The various terms of this expression are as follows:

- K is the total effort expended (in PM) in the product development and L is the product size in KLOC.
- t_d corresponds to the time of system and integration testing. Therefore, t_d can be approximately considered as the time required developing the software.

Effect of schedule change on cost

From the above expression, it can be easily observed that when the schedule of a project is compressed, the required development effort as well as project development cost increases in proportion to the fourth power of the degree of compression. It means that a relatively small compression in delivery schedule can result in substantial penalty of human effort as well as development cost. For example, if the estimated development time is 1 year, then in order to develop the product in 6 months, the total effort required to develop the product (and hence the project cost) increases 16 times.

Jensen Model

- Jensen model is very similar to Putnam model.
 - attempts to soften the effect of schedule compression on effort
 - makes it applicable to smaller and medium sized projects.
- Jensen proposed the equation:
 - $L = C_{te} t_d K^{1/2}$
 - Where,
 - C_{te} is the effective technology constant,
 - t_d is the time to develop the software, and

K is the effort needed to develop the software

Project scheduling:

Project-task scheduling is an important project planning activity. It involves deciding which tasks would be taken up when. In order to schedule the project activities, a software project manager needs to do the following:

1. Identify all the tasks needed to complete the project.
 2. Break down large tasks into small activities.
- Page
3. Determine the dependency among different activities.
 4. Establish the most likely estimates for the time durations necessary to complete the activities.
 5. Allocate resources to activities.
 6. Plan the starting and ending dates for various activities.
 7. Determine the critical path. A critical path is the chain of activities that determines the duration of the project

Work breakdown structure

- WBS is used to decompose a given task set recursively into small activities.
- It provides a notation for representing the major tasks need to be carried out in order to solve a problem.
- The root of the tree is labeled by the problem name.
- Each node of the tree is broken down into smaller activities that are made the children of the node.
- Each activity is recursively decomposed into smaller sub-activities until at the leaf level, the activities requires approximately two weeks developing.

Activity networks

- WBS representation of a project is transformed into an activity network by representing activities identified in WBS along with their interdependencies.
- An activity network shows the different activities making up a project, their estimated durations, and interdependencies.
 - Each activity is represented by a rectangular node and the duration of the activity is shown alongside each task.

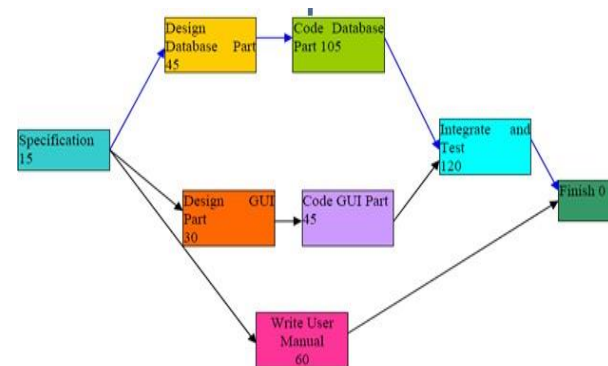


Fig. : Activity network representation of the MIS problem

Gantt chart

- Used to allocate resources to activities.
 - The resources allocated to activities include staff, hardware, and software.
- Gantt charts (named after its developer Henry Gantt) are useful for resource planning.
- It is a special type of bar chart where each bar represents an activity.
 - The bars are drawn along a time line.
 - The length of each bar is proportional to the duration of time planned for the corresponding activity.

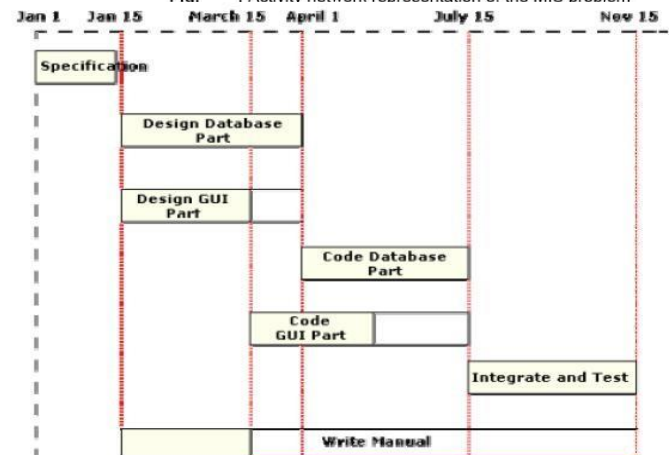


Fig. Gantt chart representation of the MIS problem

PERT chart

- PERT (Project Evaluation and Review Technique) charts consist of a network of boxes and arrows.
 - The boxes represent activities and the arrows represent task dependencies.
- It represents the statistical variations in the project estimates.
 - Thus, in a PERT chart instead of making a single estimate for each task, pessimistic, likely, and optimistic estimates are made.

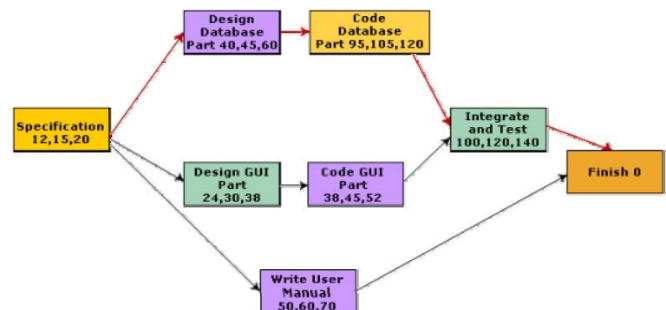


Fig. PERT chart representation of the MIS problem

- The boxes of PERT charts are usually annotated with the pessimistic, likely, and optimistic estimates for every task. Since all possible completion times between the minimum and maximum duration for every task has to be considered.

Project Organization

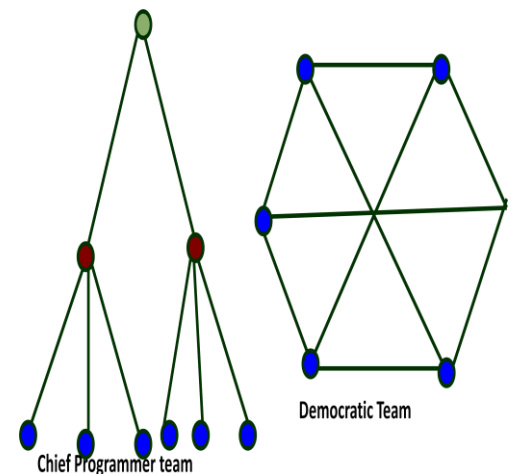
- Engineers get assigned to a project for the entire duration of the project
 - Same set of engineers carry out all the phases
- Advantages:
 - Engineers save time on learning details of every project.
 - Leads to job rotation

Team Structure

- Problems of different complexities and sizes require different team structures:
 - Chief-programmer team
 - Democratic team
 - Mixed organization

Democratic Teams

- Suitable for:
 - small projects requiring less than five or six engineers
 - research-oriented projects
- A manager provides administrative leadership:
 - at different times different members of the group provide technical leadership.
- Democratic organization provides
 - higher morale and job satisfaction to the engineers
 - Suitable for less understood problems,
 - a group of engineers can invent better solutions than a single individual.
- Disadvantage:
 - team members may waste a lot time arguing about trivial points:
 - absence of any authority in the team.



Chief Programmer Team

- A senior engineer provides technical leadership:
 - partitions the task among the team members.
 - verifies and integrates the products developed by the members.
- Works well when
 - the task is well understood
 - also within the intellectual grasp of a single individual,
 - importance of early completion outweighs other factors
 - team morale, personal development, etc.
- Chief programmer team is subject to single point failure:
 - too much responsibility and authority is assigned to the chief programmer.

Mixed Control Team Organization

- Draws upon ideas from both:
 - democratic organization and
 - chief-programmer team organization.
- Suitable for large organizations.

